Interpolation-based Transition Invariants for Proving and Disproving Termination

Abstract. Termination and nontermination of infinite-state systems are complementary problems that, despite their close connection, are typically addressed by separate techniques. The core idea of this paper is to connect termination and nontermination analysis, enabling the two to share intermediate results and guide one another. We present a new termination analysis approach based on the generation of well-founded transition invariants. It leverages Craig interpolation for transition invariant generation, capturing the structural reasons for termination. The proposed technique extends safety-based nontermination analysis, enabling it to prove both termination and nontermination within a unified framework. We implemented our approach in the Golem verification framework and evaluated it on benchmarks from the Termination Competition (TermComp). Empirical results demonstrate that combining termination and nontermination is beneficial and yields performance comparable to state-of-the-art tools.

1 Introduction

Termination and nontermination of infinite-state systems are fundamental problems in formal methods. Termination analysis is undecidable [35], yet there exist many techniques that succeed on a large class of real-world problems. Modern techniques treat termination and nontermination as separate problems, since each has a different proof procedure. The existence of a ranking function by e.g. [1,2,15,17,20,24,29,31,32,37] witnesses termination by providing a measure that strictly decreases along every transition. Alternatively, fixpoint computation, e.g. [36], and disjunctively well-founded transition invariants [3, 13, 23, 34] have been proposed for termination analysis. Nontermination techniques are predominantly based on the detection of recurrent sets — sets of states in which every state has a successor that also belongs to the set [1, 8, 9, 15, 16, 25]. There also exist alternative approaches based on Hoare-style reasoning [26], safety proving [4, 11, 33], representing infinite runs as sums of geometric series [27], or synthesis of *negative* exact loop bounds [31].

In this work, we present a unified framework for proving both termination and nontermination. This framework extends the safety-based nontermination analysis [11] (SNA), enabling it to prove termination. The extension is a novel interpolation-based method for generating disjunctively well-founded transition invariants for termination analysis. The proposed approach overapproximates terminating transition traces constructed by SNA with a Craig interpolation [14]. The interpolant is processed into a transition invariant candidate, which is used

to prove termination. This termination approach is executed in synergy with SNA. Transition invariant candidate produced by the termination analysis allows to create smaller, focused (non)termination problems, which guide the analysis of the whole transition system. Solving these problems helps to extend the transition invariant candidate and enables an efficient nontermination analysis of complex problems.

We implemented our new technique in the Golem Constrained Horn Clause solver [5]. We evaluated it on the Integer Transition Systems benchmarks from the Termination Competition [18]. Our tool both solved a large number of terminating instances and demonstrated significant improvement for nontermination analysis, compared to the SNA. The comparison with the winners of the Termination Competition 2025[1]—KoAT, LoAT [28], and T2 [8]— shows that our approach is competitive with the state-of-the-art tools and can solve unique instances, including those that were never solved by any tool in the Termination Competition before.

Overall, the contributions of this paper are as follows: (1) a novel interpolation-based method for the generation of disjunctively well-founded transition invariants, (2) a unified (non)termination analysis procedure that utilises the intermediate results for self-guidance and (3) its implementation and successful evaluation.

2 Preliminaries

This section introduces the definitions and formalisms used in the paper. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). We use X, X' to denote sets of variables (unprimed and primed). The formula defined over X is a state formula, and a formula over $X \cup X'$ is a transition formula. State formulas encode a set of states, and transition formulas define binary relations over the states. We use $Id(X, X')$ to denote the identity transition formula $\bigwedge_{x \in X} x = x'$. To denote variables over multiple states, we use superscript notation, e.g., $X^{(0)}, \ldots, X^{(n)}$.

A *transition system* is defined as $TS = \langle Init, Tr, X \rangle$, where $Init$ is a formula encoding the initial states of the system, Tr is a transition formula, encoding the transition relation, and X is the set of state variables. In the context of transition system $X' = \{x' | x \in X\}$ is a set of next state variables.

With respect to transition system, a *transition trace* is a formula encoding multiple consecutive transitions: $Tr^n(X^{(0)}, \ldots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \cdots \wedge Tr(X^{(n-1)}, X^{(n)})$ (we assume that $Tr^0 = \top$). A *safety problem* is $P = \langle Init, Tr, Bad, X \rangle$, where Bad is a formula representing states that violate the safety property.

Safety verification amounts to determining whether there exists a transition trace that reaches a Bad state from some initial state. If TS is safe, then there exists an inductive invariant $Inv(X)$ such that (1) $\forall X. Init(X) \rightarrow Inv(X)$, (2)

[1] `https://termination-portal.org/wiki/Termination_Competition_2025`.

$\forall X, X'.Inv(X) \wedge Tr(X, X') \rightarrow Inv(X')$, and (3) $\forall X.Inv(X) \rightarrow \neg Bad(X)$. We use SafetyCheck (P) to denote a safety verification procedure that returns a tuple $\langle \text{SAFE}, Inv(X) \rangle$ if TS is safe ($Inv(X)$ is an inductive invariant). If TS is unsafe it returns $\langle \text{UNSAFE}, Tr^n(X^{(0)}, \ldots, X^{(n)}) \rangle$, s.t. $\exists X^{(0)}, \ldots, X^{(n)}$[2].

$$Init(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge Bad(X^{(n)})$$

A transition system is called **nonterminating** if there exists an initial state from which exists an infinite sequence of states, meaning:

$$\exists X^{(0)}, X^{(1)}, \ldots . Init(X^{(0)}) \wedge \forall i \in \mathbb{N}. Tr(X^{(i)}, X^{(i+1)})$$

Definition 1 (Recurrent Set). *For a transition system $\langle Init, Tr, X \rangle$, a formula $N(X)$ is a recurrent set if: (1) $\forall X.N(X) \rightarrow \exists X'.Tr(X, X') \wedge N(X')$, (2) $\exists X.N(X) \wedge Init(X)$.*

Transition system is nonterminating if there exists a *recurrent set*. Intuitively, every state in a recurrent set has a successor that also lies in the recurrent set, and the recurrent set contains some of the initial states.

A TS is called **terminating** if for every initial state there does not exist an infinite sequence of reachable states. Transition system $\langle Init, Tr, X \rangle$ terminates, if the following holds:

$$\neg \exists X^{(0)}, X^{(1)}, \ldots . Init(X^{(0)}) \wedge \forall i \in \mathbb{N}. Tr(X^{(i)}, X^{(i+1)})$$

Definition 2 (Transition invariant). *For a transition system $\langle Init, Tr, X \rangle$, $TrInv(X, X')$ is a transition invariant if: $\forall X, X'.R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $Tr^+(X, X')$ is the transitive closure of $Tr(X, X')$ and $R(X)$ is a formula encoding all states reachable in TS.*

Intuitively, a transition invariant is a formula that overapproximates any possible number of transitions in the transition system.

Definition 3 (Well-founded Relation). *A binary relation $W \subseteq D \times D$ over a domain D is* well-founded *if there exists no infinite descending chain:*

$$\nexists\, d_0, d_1, d_2, \cdots \in D.\ \forall i \in \mathbb{N}.\ (d_i, d_{i+1}) \in W$$

A relation $W(X, X')$ can be proven to be well-founded by synthesizing a *ranking function* $f(X)$ that is bounded from below and decreases by some $\delta > 0$ along every step of W [29]: $\forall X, X'.\ W(X, X') \rightarrow f(X) \geq 0 \wedge f(X) \geq f(X') + \delta$.

Termination can be proven using transition invariants [30]. If a transition invariant is a finite union of well-founded relations (for example, if each relation admits a ranking function), then the transition system terminates.

Definition 4 (Disjunctively well-founded relation). *A relation $K(X, X') = \bigcup_{i=0}^{n} W_i(X, X')$ is called disjunctively well-founded if every relation $W_i(X, X')$ is well-founded.*

[2] It is a shorthand notation for $\exists X^{(0)}, \ldots, \exists X^{(n)}$

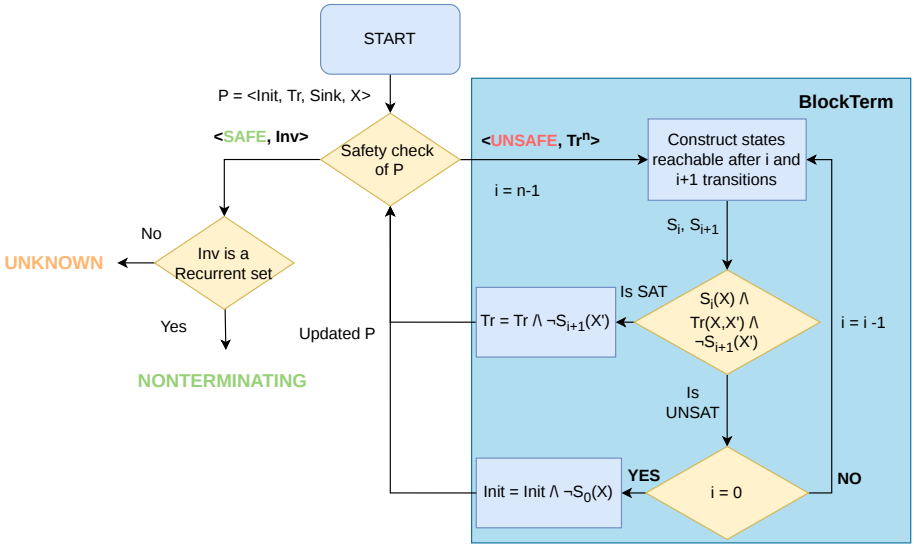


Fig. 1. Execution flow of Safety-based Nontermination analysis, cf. [11]

Given two formulas A and B where $A \wedge B$ is unsatisfiable, a Craig interpolant [14] for A and B is a formula I, where: (1) $A \rightarrow I$; (2) $I \wedge B$ is unsatisfiable; and (3) I only contains the symbols, common to both A and B. Craig interpolants are widely used as overapproximations. We write $\textsc{Itp}(A, B)$ to denote a Craig interpolant.

2.1 Proving Nontermination via Safety

Many techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs recurrent sets via inductive invariants [11]. It was introduced in the context of proving nontermination of programs with loops, and can be extended to transition systems.

Definition 5 (Sink states). *For a transition system $\langle Init, Tr, X\rangle$, a state s is a sink state, if $\neg\exists s'.Tr(s, s')$. We use $Sink(X)$ to denote the formula encoding all sink states in the transition system: $\forall X.Sink(X) \leftrightarrow \neg\exists X'.Tr(X, X')$*

Algorithm 1 for Safety-based Nontermination analysis (SNA) takes a safety problem $P = \langle Init, Tr, Sink, X\rangle$, where sink states are constructed from TS using quantifier elimination (QE): $Sink(X) = \neg\text{QE}(\exists X'.Tr(X, X'))$. Throughout the paper we assume that QEis exact. As an output, it returns NONTERM if the system is nonterminating, and UNKNOWN, if the algorithm cannot conclude the analysis. Figure 1 illustrates the execution flow of Safety-based Nontermination. As a preprocessing step, the sink states are removed from initial

Algorithm 1: Safety-based Nontermination Analysis (SNA), cf. [11]

```
   Input  : Safety problem P = ⟨Init, Tr, Sink, X⟩
   Output: NONTERM | UNKNOWN
1  while ⊤ do
2  |  ⟨res, φ = Tr^n or Inv⟩ ← SafetyCheck(P)
3  |  if res = UNSAFE then
4  |  |  P ← BlockTerm(P, φ)
5  |  else
6  |  |  if ∀X.φ(X) → ∃X'.Tr(X, X') and ∃X.Init(X) ∧ φ(X) then
7  |  |  |  return NONTERM
8  |  |  return UNKNOWN
```

states, $Init \leftarrow Init \wedge \neg Sink$, since no transition can be taken from them (it is also applied to the algorithms later in the paper).

SNA iteratively solves increasingly refined safety problems until the given sink states are proven unreachable. In every iteration, if there exists a trace reaching sink states (line 3), BlockTerm (Algorithm 2) refines the safety problem P by blocking the states that deterministically lead to the sink states, updating the safety problem. BlockTerm is described in detail below. When the original sink states are no longer reachable, SNA checks if the computed inductive invariant $Inv(X)$ is a recurrent set (line 6). This amounts to deciding satisfiability of the following formula:

$$Inv(X) \wedge \neg \exists X'. Tr(X, X').$$

If $Inv(X)$ is a recurrent set, then the transition system is nonterminating. Otherwise, SNA returns UNKNOWN.

Lemma 1. *If Algorithm 1 returns NONTERM, then the transition system is nonterminating, cf. [11].*

The BlockTerm procedure takes as input a safety problem P and a trace formula $Tr^n(X^{(0)}, \ldots, X^{(n)})$ such that $Init(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge Sink(X^{(n)})$ is satisfiable. The algorithm iteratively analyses the trace from the last transition backwards. Using QEAlgorithm 2 computes S_i and S_{i+1}, encoding the states that are reachable after i and $i+1$ transitions from $Init$ (lines 2, 3). Then, at line 4 it checks if $S_i(X) \wedge Tr(X, X') \wedge \neg S_{i+1}(X')$ is satisfiable. Intuitively, with this query, the algorithm checks if it is possible to take an alternative transition at step i, one that does not lead to the sink states. If it is possible, the algorithm blocks the states deterministically leading to the sink states by excluding them from the transition relation (line 5). If the whole trace is deterministic (it is not possible to take an alternative transition along the trace), BlockTerm blocks the subset of initial states deterministically leading to sink (line 7).

Lemma 2. *Let $P = \langle Init, Tr, Sink, X\rangle$ be a safety problem where Sink are sink states, and let $\langle Init', Tr', Sink, X\rangle = \textsc{BlockTerm}(P, Tr^n)$. Then every infinite*

Algorithm 2: BLOCKTERM (P, Tr^n), cf. [11]

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$, terminating trace $Tr^n(X^{(0)}, \ldots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \cdots \wedge Tr(X^{(n-1)}, X^{(n)})$

Output: Refined safety problem P'

Data : $S_0, \ldots, S_n$ - formulas which encode states within trace reachable in 0, ..., n transitions

for $i = n-1; i >= 0; i \leftarrow i-1$ **do**
  $S_{i+1}(X) \leftarrow \mathrm{QE}(\exists X^{(0)}, \ldots, X^{(i)}, X^{(i+2)}, \ldots, X^{(n)}.$
    $Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))[X^{(i+1)} \mapsto X]$
  $S_i(X) \leftarrow \mathrm{QE}(\exists X^{(0)}, \ldots, X^{(i-1)}, X^{(i+1)}, \ldots, X^{(n)}.$
    $Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))[X^{(i)} \mapsto X]$
  if *SAT ?* $S_i(X) \wedge Tr(X, X') \wedge \neg S_{i+1}(X')$ **then**
    $Tr'(X, X') \leftarrow Tr(X, X') \wedge \neg S_{i+1}(X')$
    return $\langle Init, Tr', Sink, X\rangle$
$Init'(X^{(0)}) \leftarrow Init(X^{(0)}) \wedge \neg \mathrm{QE}(\exists X^{(1)}, \ldots, X^{(n)}. Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))$

return $\langle Init', Tr, Sink, X\rangle$

run of $\langle Init, Tr, X\rangle$ *is an infinite run of* $\langle Init', Tr', X\rangle$ *cf. [11] (proof is given in Appendix D).*

Example 1. Consider a transition system $TS = \langle Init, Tr, X\rangle$ with $X = \{x\}$, $Init(X) \triangleq \top$, and $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x \neq 0$. The sink states are $Sink(X) \triangleq x = 0$, and the initial safety problem is $P = \langle Init, Tr, Sink, X\rangle$. This TS is nonterminating, since x infinitely decreases.

First, preprocessing removes sink states from the initial states: $Init(X) \leftarrow x \neq 0$. Then, Algorithm 1 updates the transition relation via BLOCKTERM: $Tr(X, X') \leftarrow Tr(X, X') \wedge x' \neq 0$. The safety check of the updated problem concludes SAFE, with $Inv(X) \triangleq x \neq 0$. Invariant is a recurrent set, and algorithm returns NONTERM.

Motivation for transition invariant-based termination analysis. Consider a small modification (replacing $x \neq 0$ with $x > 0$): $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x > 0$. This renders TS terminating since x is strictly positive and decreases every step. However, Algorithm 1 cannot detect termination by design, and it would return UNKNOWN.

Let us examine the execution of Algorithm 1 on this example. First, preprocessing removes sink states from initial states: $Init(X) \leftarrow x > 0$. In the first iteration, SAFETYCHECK (P) returns UNSAFE with a trace $Tr^1(X^{(0)}, X^{(1)})$ satisfying $\exists X^{(0)}, X^{(1)}. Init(X^{(0)}) \wedge Tr^1(X^{(0)}, X^{(1)}) \wedge Sink(X^{(1)})$. Our key insight is to generalise such terminating traces into a formula describing an entire class of terminating behaviours. If this generalisation yields a disjunctively well-founded transition invariant over all reachable states, it witnesses termination. Particularly, $TrInv(X, X') \triangleq x' < x \wedge x > 0 \wedge x' > -2$ is a well-founded transition invariant that proves the termination of TS. Section 3 presents the algorithm that automates this generalisation via Craig interpolation.

3 Termination Analysis via Interpolation-based Transition Invariant Generation

We present a new termination analysis approach that leverages safety verification to generate a well-founded transition invariant via interpolation. At each iteration, our procedure constructs a trace from the initial states to a sink state. A candidate well-founded transition invariant is then constructed based on the overapproximation of the terminating trace. If the candidate is valid for all reachable states (meaning it is a transition invariant), termination is concluded. The interpolation-based approach enables efficient generation of a transition invariant guided by the termination conditions.

3.1 Interpolation-based Generation of Well-Founded Transition Invariants

The core of our approach is TrInvGen, a procedure for the automated interpolation-based generation of well-founded transition invariants. Given a transition system and a terminating trace, it constructs a disjunctively well-founded approximation of the trace. If the resulting formula constitutes a transition invariant over all reachable states, it witnesses termination.

The procedure consists of five steps: (1) Computing NT^n, the formula encoding states from which non-sink states can be reached in n transitions. NT^n is used to construct the interpolant. The negation of NT^n encodes the states from which *Sink* is guaranteed to be reached in at most n transitions. (2) Constructing an over-approximation of the terminating trace via interpolation. The usage of sink states allows the interpolant to capture the reason for termination. (3) Converting the interpolant into an underapproximating DNF formula. This enables efficient ranking function synthesis for each disjunct. (4) Filtering disjuncts, preserving only disjuncts that encode well-founded relations. These disjuncts are used to construct a candidate transition invariant. Only well-founded relations are preserved, as transition invariant needs to be disjunctively well-founded to witness termination. (5) Computing the formula $C(X)$, encoding states for which candidate transition invariant is valid. Transition invariant needs to be valid only over the reachable states to conclude termination. Therefore, if all reachable states satisfy $C(X)$, then TS is terminating. The overall flow of the procedure is illustrated in Figure 2 and described in Algorithm 3.

Algorithm 3 takes as input a safety problem $P = \langle Init, Tr, Sink, X\rangle$, a transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, and a transition invariant formula $TrInv(X, X')$. It returns a tuple of formulas $\langle TrInv, C\rangle$, where $TrInv(X, X')$ is a disjunctively well-founded transition invariant candidate, and $C(X)$ is a formula encoding the states for which *TrInv* is a valid transition invariant. It proceeds in the following steps:

Step 1: *Computing NT^n (line 1).* $NT^n(X)$ encodes all states from which a non-sink state is reachable in exactly n transitions. The following property defines it: $\forall X^{(0)}.NT^n(X^{(0)}) \leftrightarrow$

$$\exists X^{(1)}, \ldots, X^{(n)}.Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

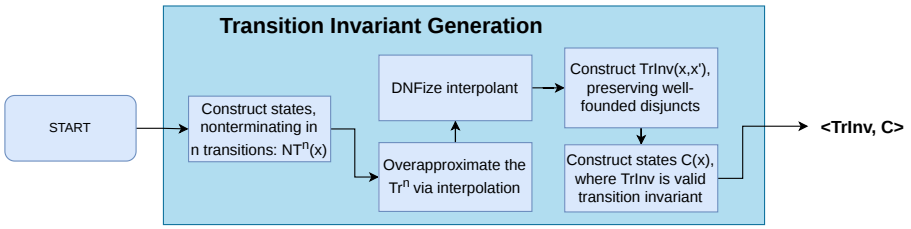


Fig. 2. Execution Flow of Interpolation-based Transition Invariant Generation

Algorithm 3: TrInvGen(P, Tr^n, $TrInv$).

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$, transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, candidate transition invariant $TrInv(X, X')$

Output: Updated transition invariant $TrInv(X, X')$ and formula $C(X)$ encoding states covered by transition invariant

1 $NT^n(X^{(0)}) \leftarrow \text{QE}(\exists X^{(1)}, \ldots, X^{(n)}.\ Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)}))$
2 $itp(X, X') \leftarrow \text{Itp}(Tr^n(X, \ldots, X'), \neg NT^n(X) \wedge \neg Sink(X'))$
3 $Cs(X, X') \leftarrow \text{ToDNF}(itp)$
4 $TrInv \leftarrow TrInv \vee \bigvee_i \{d_i \in Cs \mid \exists \text{ ranking function } f(X)\}$
5 $C(X) \leftarrow \neg\text{QE}(\exists X', X''.$
$\big(Id(X, X') \vee TrInv(X, X')\big) \wedge Tr(X', X'') \wedge \neg TrInv(X, X''))$
6 **return** $\langle TrInv, C\rangle$

NT^n is constructed with quantifier elimination, eliminating all variables from the formula except for $X^{(0)}$. Construction of NT^n enables the overapproximation of the trace.

Example 2. Consider $TS = \langle Init, Tr, X\rangle$ with $Init \triangleq x > 0$, $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge (x > 0 \vee x < -10)$, $X \triangleq \{x\}$. The sink states are $Sink(X) \triangleq x \leq 0 \wedge x \geq -10$. For $n = 1$, quantifier elimination yields $NT^1(X) = x > 1 \vee x < -10$: states from which $\neg Sink$ is reachable in one transition.

Step 2: *Interpolation (line 2).* The following formula is unsatisfiable by construction since any state that satisfies $\neg NT^n$ must reach $Sink$ within at most n steps:

$$\neg NT^n(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

This allows us to compute a *Craig interpolant* that overapproximates the trace Tr^n. Crucially, the usage of the sink states for the interpolant construction allows the interpolant to capture the reason *why* the particular trace is terminating.

Step 3: *DNF underapproximation (line 3).* For a transition invariant to witness termination, it must be disjunctively well-founded: every disjunct must encode a well-founded relation. Converting the interpolant to DNF would simplify the well-foundedness check for separate disjuncts. However, complete DNF

conversion can cause an exponential blowup in formula size and take a significant amount of time. Moreover, even a complete DNF is not guaranteed to yield a transition invariant.

Instead, our algorithm follows a lazy approach: it constructs an *under-approximating* DNF formula on demand. Specifically, the algorithm constructs disjuncts as follows: (1) the SMT solver produces a model that satisfies *itp*; (2) the literals satisfied by this model are extracted and conjoined to form a single disjunct; (3) this disjunct is blocked from *itp* and the process repeats until some m disjuncts are constructed. The result is a formula:

$$Cs(X, X') \triangleq \bigvee_{i=0}^{m} d_i$$

where each d_i is a conjunction, and $Cs(X, X') \rightarrow itp(X, X')$. This approach significantly reduces formula size and speeds up both the DNF construction and subsequent well-foundedness check, while preserving correctness: in the worst case, *Cs* fails to be a transition invariant, but this is also possible even with a complete DNF conversion.

Step 4: *Filtering Well-Founded Disjuncts (line 4).* For each disjunct $d(X, X') \in Cs(X, X')$, the algorithm checks if $d(X, X')$ encodes a well-founded relation. Our technique synthesizes linear ranking functions automatically following [29] for checking the existence of a ranking function. The disjuncts admitting a ranking function are preserved, the rest are discarded. The resulting formula is disjoined with the input candidate transition invariant *TrInv*.

Step 5: *Synthesis of the states for which transition invariant is valid (line 5).* If *TrInv* is a transition invariant over all reachable states, then *TS* terminates, since every disjunct encodes a well-founded relation. To verify this, the procedure computes the formula $C(X)$ encoding the states over which *TrInv* is a valid transition invariant (including sink states, as any candidate invariant is valid over *Sink*).

Definition 6 (Coverage formula). *Given a transition system $TS = \langle Init, Tr, X\rangle$ and a formula $TrInv(X, X')$, a formula $C(X)$ is called coverage, if the following holds $\forall X, X', X''$:*

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$
$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

That is, TrInv is a transition invariant over states that satisfy $C(X)$.

Similarly to NT^n, $C(X)$ is constructed by applying quantifier elimination, negating the states for which *TrInv* is not a transition invariant. It is a coverage formula by construction.

Example 3. Continuing Example 2, suppose the procedure constructs an interpolant $itp(X, X') \triangleq x' < x \wedge ((x > 0 \wedge x' > -2) \vee x < -10)$. Afterwards, algorithm would construct DNF form: $(x' < x \wedge x > 0 \wedge x' > -2) \vee (x' < x \wedge x < -10)$, and since the second disjunct is not well-founded, it would be dropped, resulting in

$TrInv(X, X') \triangleq x' < x \wedge x > 0 \wedge x' > -2$. Quantifier elimination at line 5 then returns $C(X) \triangleq x \geq -10$.

Note. If $\neg C(X)$ is unreachable, then $TrInv(X, X')$ is a disjunctively well-founded transition invariant over all reachable states, and witnesses termination of the transition system. For illustration, the transition invariant in Example 3 is sufficient to prove termination since the states satisfying $\neg C(X) \triangleq x < -10$ cannot be reached within TS.

Lemma 3. *Let $TS = \langle Init, Tr, X\rangle$ be a transition system and let $C(X)$ be a formula such that $\neg C(X)$ is unreachable in TS. If $TrInv(X, X')$ is a finite disjunction of formulas encoding well-founded relations and satisfies $\forall X, X', X''$:*

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$
$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

then TS is terminating.

Proof. Full proof is provided in Appendix A. □

Remark: Soundness of TERM does not depend on the interpolant: Lemma 3 only requires that the disjuncts kept at line 4 are well-founded and that C is a coverage formula, both of which are checked.

3.2 Interpolation-based Termination Analysis

This subsection presents the full termination analysis procedure using the interpolation-based transition invariant generation. It integrates TrInvGen into the main loop of Algorithm 1. At each iteration, instead of only blocking terminating states, the algorithm also constructs a transition invariant candidate. The overall flow is shown in Figure 3.

The Algorithm 4 takes as input a safety problem $P = \langle Init, Tr, Sink, X\rangle$, where $Init$ and Tr encode the initial states and transition relation, $Sink$ is the formula encoding sink states, and X is the set of variables over which the transition system is defined. As an output, the algorithm returns NONTERM if TS is nonterminating, TERM if it is terminating, and UNKNOWN if the procedure cannot determine the final result. The procedure maintains a candidate transition invariant $TrInv(X, X')$. During the execution, $TrInv$ is extended with new disjuncts, which are generated by the TrInvGen procedure, and encode well-founded relations. Changes to the original SNA are at lines 5, 6, and 8.

Every iteration of Algorithm 4, similarly to the Algorithm 1, begins with the safety check at line 3. The algorithm handles two possible cases:

UNSAFE A terminating trace Tr^n exists. The algorithm calls TrInvGen to extend $TrInv$ and compute the coverage formula $C(X)$ (line 5). Then, it checks if $\neg C(X)$ is reachable from the initial states (line 6). If $\neg C(X)$ is reachable, the algorithm refines the safety problem by blocking the states that deterministically lead to termination via BlockTerm, and the loop continues. Otherwise, $TrInv$ is a transition invariant over all reachable states, and the algorithm concludes termination.

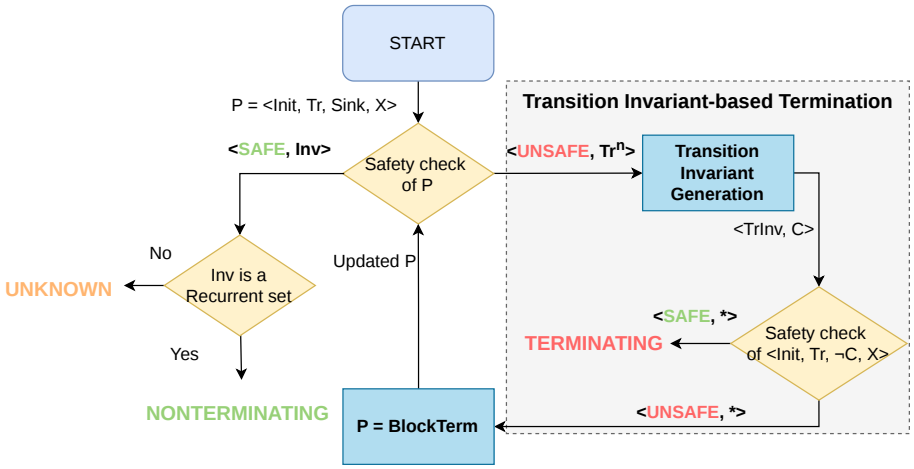


Fig. 3. Execution Flow of Interpolation-based Transition Invariant Generation (New components are placed in a grey block, lines 5,6,8).

Algorithm 4: Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$
Output: TERM | NONTERM | UNKNOWN
Data : $TrInv$ - formula that encodes transition invariant

1 $TrInv \leftarrow \bot$
2 **while** $\top$ **do**
3 $\quad \langle res, \phi = Tr^n \text{ or } Inv\rangle \leftarrow$ SafetyCheck(P)
4 $\quad$ **if** $res = UNSAFE$ **then**
5 $\quad\quad \langle TrInv, C\rangle \leftarrow$ TrInvGen$(P, \phi, TrInv)$
6 $\quad\quad \langle res', _\rangle \leftarrow$ SafetyCheck$(\langle Init, Tr, \neg C, X\rangle)$
7 $\quad\quad$ **if** $res' = UNSAFE$ **then** $P \leftarrow$ BlockTerm(P, ϕ)
8 $\quad\quad$ **else return** TERM
9 $\quad$ **else**
10 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr(X, X') \wedge \phi(X')$ *and* $\exists X.Init(X) \wedge \phi(X)$ **then**
11 $\quad\quad\quad$ **return** NONTERM
12 $\quad\quad$ **return** UNKNOWN

SAFE No sink state is reachable, and there exists an inductive invariant $Inv(X)$. If $Inv(X)$ is a recurrent set, the system is nonterminating. Otherwise, the algorithm returns UNKNOWN as the analysis is inconclusive.

Example 4. Consider TS from Example 2 with $Init \triangleq x > 0$, $Tr \triangleq (x' = x-1 \vee x' = x-2) \wedge (x > 0 \vee x < -10)$, $X \triangleq \{x\}$. Preprocessing computes $Sink \triangleq x \leq 0 \wedge x \geq -10$. Then, the algorithm is called with $P = \langle Init, Tr, Sink, X\rangle$.

The safety check detects a trace Tr^1 reaching $Sink$. TrInvGen produces a transition invariant candidate $TrInv(X, X') \triangleq x' < x \wedge x > 0 \wedge x' > -2$ and

the formula $C(X) \triangleq x \geq -10$. The safety check for $\langle \mathit{Init}, \mathit{Tr}, \neg C(X), X \rangle$ returns SAFE, and the algorithm concludes that the transition system is terminating.

Limitation of the approach. Consider a different TS, with $X = \{x, u\}$, $\mathit{Init}(X) \triangleq x > 0 \wedge u \leq 10$, and $\mathit{Tr}(X, X') \triangleq ((x' = x - 1 \wedge u' = u) \vee (x' = x + 1 \wedge u' = u + 1 \wedge u < 10)) \wedge x > 0 \wedge u \leq 10$. Intuitively, each transition either decrements x (and keeps u unchanged), or increments both x and u while $u < 10$. Once $u = 10$, only the decrementing transition is possible, and x decreases monotonically to zero. Hence TS is terminating. The sink states are $\mathit{Sink}(X) \triangleq x \leq 0 \vee u > 10$.

Algorithm 4 detects a trace $\mathit{Tr}^1(X^{(0)}, X^{(1)})$ reaching *Sink* states. One possible interpolant is $\mathit{itp}(X, X') \triangleq (x' < x \wedge x > 0 \wedge u' = u) \vee (x > 0 \wedge u < 10)$; its second disjunct is not well-founded, so TrInvGen constructs $\mathit{TrInv}(X, X') \triangleq x' < x \wedge x > 0 \wedge u' = u$ and the coverage formula $C(X) \triangleq x \leq 0 \vee u \geq 10$. The safety check for $\langle \mathit{Init}, \mathit{Tr}, \neg C, X \rangle$ returns UNSAFE, since initial states with $u < 10$ satisfy $\neg C$. BlockTerm then blocks the states that deterministically reach *Sink*: $\mathit{Tr}(X, X') \leftarrow \mathit{Tr}(X, X') \wedge \neg(x' = 0 \wedge u' \leq 10)$.

The algorithm calls SafetyCheck(P), which returns SAFE with inductive invariant $\mathit{Inv}(X) \triangleq x > 0 \wedge u \leq 10$. However, *Inv* is not a recurrent set (the state $x = 1, u = 10$ has no successor), so Algorithm 4 returns UNKNOWN.

The root cause why the algorithm cannot prove termination is that *TrInv* does not cover states with $u < 10$. It captures why the system terminates when $u = 10$, but not the global argument: u grows until at most $u = 10$, after which x decreases monotonically. A complete termination proof requires a *second* disjunct covering the $u < 10$ region. In Section 4, we present an extension that focuses on the state space for which the transition invariant candidate is not valid, aiming to extend the transition invariant.

Correctness: Below, the soundness of the Algorithm 4 is discussed. The proofs of Lemma 4 and Lemma 5 are provided in Appendix B.

Lemma 4. *If Algorithm 4 returns TERM, then the transition system* $\langle \mathit{Init}, \mathit{Tr}, X \rangle$ *is terminating.*

Lemma 5. *If Algorithm 4 returns NONTERM, then the transition system* $\langle \mathit{Init}, \mathit{Tr}, X \rangle$ *is nonterminating.*

4 Extended Interpolation-based Transition Invariant Generation for Proving and Disproving Termination

Algorithm 4 is already capable of proving termination efficiently through the construction of transition invariants. This section describes an extension of the algorithm that aims to improve interaction between termination and nontermination analyses. Additionally, we present further extensions to the original SNA algorithm that improve overall performance. The high-level flow of the extended algorithm can be seen in Figure 4.

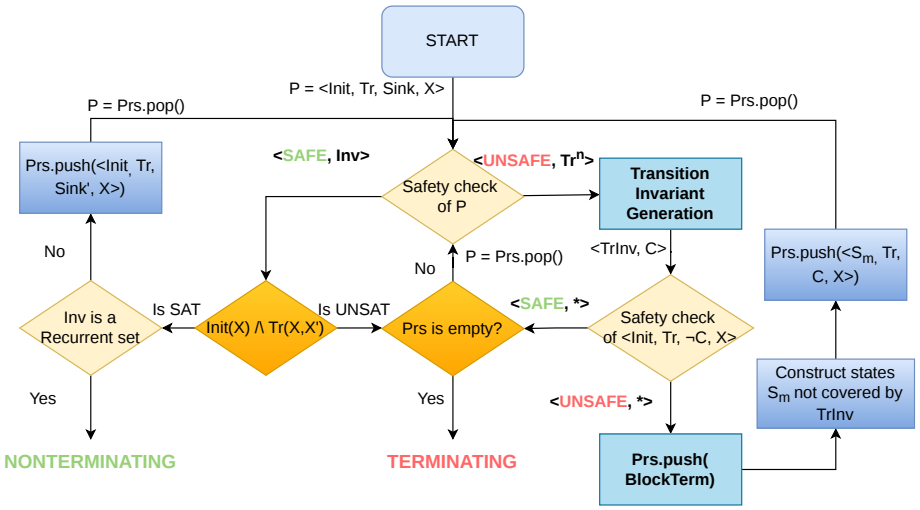


Fig. 4. Extended Interpolation-based Termination Analysis (new components are highlighted in dark orange and dark blue colors).

4.1 Analysis of States for which Transition Invariant is not Valid

A transition invariant candidate can be used to guide the termination analysis. Particularly, Algorithm 4 constructs the formula $C(X)$ encoding states for which the transition invariant is valid. If $\neg C(X)$ is reachable, there exist states for which the transition invariant is not valid, and which potentially can lead to nontermination. The intuition is to compute the reachable states S_m within $\neg C(X)$ (which corresponds to line 11 of Algorithm 5). Then, it is possible to run (non)termination check for these particular states within TS (line 12).

In the case if any transition trace from S_m eventually reaches C, the algorithm produces a witness transition invariant that extends the transition invariant of the original system, potentially making it valid for more states. If nontermination is proved for S_m, then the original TS is also nonterminating, since S_m is reachable from the initial states.

4.2 Overall Algorithm

Algorithm 5 extends Algorithm 4 to enable more efficient (non)termination analysis. First, in addition to maintaining the transition invariant, the algorithm maintains a stack of safety problems to be analyzed: Prs (line 2). At each iteration, the algorithm pops a safety problem $P' = \langle Init, Tr, Sink, X\rangle$ from the stack and analyzes it (line 4). The stack allows the algorithm to focus on (non)termination analysis of particular reachable states within TS. The original safety problem always remains in the Prs stack.

The postprocessing phase of transition invariant construction is the main change to Algorithm 4. When $\neg C$ is reachable, the extended algorithm performs

Algorithm 5: Extended Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem P
Output: TERM | NONTERM
Data : *TrInv* - candidate transition invariant, *Prs* - stack of safety problems

1 $TrInv \leftarrow \bot$
2 $Prs \leftarrow [P]$
3 **while** $\top$ **do**
4 $\quad P' \equiv \langle Init, Tr, Sink, X\rangle \leftarrow Prs.pop()$
5 $\quad \langle res, \phi\rangle \leftarrow$ SafetyCheck(P')
6 $\quad$ **if** $res = UNSAFE$ **then**
7 $\quad\quad \langle TrInv, C\rangle \leftarrow$ TrInvGen$(P', \phi, TrInv)$
8 $\quad\quad \langle res', \phi'\rangle \leftarrow$ SafetyCheck$(\langle Init, Tr, \neg C, X\rangle)$
9 $\quad\quad$ **if** $\langle res', \phi'\rangle = \langle UNSAFE, Tr^m\rangle$ **then**
10 $\quad\quad\quad Prs.push($BlockTerm$(P', \phi))$
11 $\quad\quad\quad S_m(X) \leftarrow QE(\exists X^{(0)}, \ldots, X^{(m-1)}.Init(X^{(0)}) \wedge \phi'(X^{(0)}, \ldots, X^{(m)}) \wedge \neg C(X^{(m)}))[X^{(m)} \mapsto X]$
12 $\quad\quad\quad Prs.push(\langle S_m, Tr, C, X\rangle)$
13 $\quad\quad$ **else**
14 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then return** TERM
15 $\quad\quad\quad$ **else continue**
16 $\quad$ **else**
17 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr(X, X') \wedge \phi(X')$ *and* $\exists X.Init(X) \wedge \phi(X)$ **then**
18 $\quad\quad\quad$ **return** NONTERM
19 $\quad\quad$ **if** *UNSAT? Init*$(X) \wedge Tr(X, X')$ **then**
20 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then return** TERM
21 $\quad\quad\quad$ **else continue**
22 $\quad\quad Sink'(X) \leftarrow \neg QE(\exists X'.Tr(X, X'))$
23 $\quad\quad Prs.push(\langle Init \wedge \neg Sink', Tr, Sink', X\rangle)$

three actions: (1) it pushes the BlockTerm-refined problem (line 10); (2) it computes a formula encoding reachable states, for which transition invariant candidate is not valid (line 11); and (3) it pushes a new subproblem to analyze for termination (line 12), with S_m as the initial states. This allows the algorithm to synthesize additional termination arguments or prove nontermination of particular reachable states.

If $\neg C$ is not reachable, *TrInv* proves termination of the currently analyzed *TS* (the last *TS* popped from the stack), and the number of remaining subproblems decreases. If the stack is empty, Algorithm 5 returns TERM. The algorithm incorporates two further extensions.

Simple termination check (line 19). When the safety check returns SAFE and $\phi(X)$ is not a recurrent set, the algorithm additionally checks whether it is possible to take a transition from the initial states by querying:

$$Init(X) \wedge Tr(X, X')$$

If this formula is unsatisfiable, then no transition can be taken from the initial states (or all transitions are blocked, meaning all reachable states deterministically lead to termination), and the system is trivially terminating.

Refinement of Sink states (lines 22, 23). When $\phi(X)$ is not a recurrent set, this is often because BlockTerm has eliminated all transitions leading to *Sink*. This means that the produced ϕ contains sink states due to the updated Tr. New formula encoding sink states is computed as:

$$Sink'(X) = \neg QE(\exists X'.\, Tr'(X, X'))$$

Formula $Sink'$ replaces *Sink* in the safety problem, enabling the analysis to proceed instead of returning UNKNOWN.

Correctness: Below, the soundness of the Algorithm 5 is discussed. Proof for Lemma 6 is given in the Appendix C.

Lemma 6. *Let $P_0 = \langle Init_0, Tr_0, Sink_0, X\rangle$ be the (preprocessed) input of Algorithm 5. At the start of every iteration, the bottom element $B = \langle Init_B, Tr_B, Sink_B, X\rangle$ of Prs satisfies: (1) no $Sink_B$ state has a Tr_B-successor, and (2) every infinite run of $\langle Init_0, Tr_0, X\rangle$ is an infinite run of $\langle Init_B, Tr_B, X\rangle$.*

Lemma 7. *If Algorithm 5 returns TERM, then the original transition system $\langle Init, Tr, X\rangle$ is terminating.*

Proof. TERM is returned only at lines 14 and 20, both when $Prs = \emptyset$ after the pop, i.e., when the analyzed problem is the bottom element B of Lemma 6. At line 14, $\neg C$ is unreachable in $\langle Init_B, Tr_B, X\rangle$, C is a coverage formula of *TrInv* w.r.t. Tr_B, and every disjunct of *TrInv* admits a ranking function (regardless of the subproblem that produced it); by Lemma 3, $\langle Init_B, Tr_B, X\rangle$ terminates. At line 20, $Init_B \wedge Tr_B$ is unsatisfiable, so $\langle Init_B, Tr_B, X\rangle$ terminates trivially. In both cases, by Lemma 6(2) the original system is terminating. □

Lemma 8. *If Algorithm 5 returns NONTERM, then the original transition system $\langle Init, Tr, X\rangle$ is nonterminating.*

Proof. NONTERM is returned at line 18, where $\phi(X)$ is a recurrent set for the currently considered *TS* $\langle Init', Tr', X\rangle$. By Lemma 1, there exists a recurrent set in the transition system $\langle Init', Tr', X\rangle$, and since $Tr'(X, X') \rightarrow Tr(X, X')$ and $Init'$ is reachable from *Init* (or it is *Init*), this recurrent set witnesses nontermination of *TS*. □

5 Implementation and Evaluation

5.1 Implementation

We implemented the prototypes of Algorithms 1, 4, and 5 in Golem; we refer to these implementations as SNA, ItpTig, and ItpTig+. Golem was chosen because it supports multiple different safety verification procedures, such

Table 1. Comparison of ItpTig+ with SNA and ItpTig. *Unique*: instances solved by neither of the other two configurations.

Algorithm	Total	Terminating	Nonterminating	Unique
SNA	343	0	343	7
ItpTig	521	186	335	7
ItpTig+	805	358	447	284

as Property-Directed Reachability [7] (PDR), Transition Power Abstraction [6] (TPA), and Property-Directed k-Induction [22] (PDKIND). Additionally, we used quantifier elimination, already implemented in Golem. For SMT queries and interpolation, our implementation uses the OpenSMT solver [10,21]. OpenSMT is integrated within Golem and supports interpolation.

5.2 Evaluation

We compared ItpTig+ against SNA and ItpTig to evaluate the effect of combining termination and nontermination analyses. Additionally, we evaluated ItpTig+ against several state-of-the-art tools: LoAT [28] (version eacb74b), KoAT [28] (version 41cb681), T2 [8] (version 10f1373). The evaluation focused on termination analysis of Integer Transition Systems. Experiments were conducted on an Ubuntu 20.04 machine with an AMD Epyc 7742 64-core CPU and 8x32 GiB RAM. Each benchmark was executed on 2 cores with 2 GiB of allocated RAM. Our evaluation addressed the following research questions:

- **RQ1:** How does extending SNA with the termination procedure affect verification performance?
- **RQ2:** How does ItpTig+, as part of the Golem portfolio, compare to state-of-the-art tools?

The benchmark suite of Integer Transition Systems was taken from the Termination Competition [18]. Overall, there are 1222 benchmarks (538 nonterminating, 665 terminating, and 19 unknown). We excluded 42 trivially terminating benchmarks, whose termination can be detected by simple syntactic checks; all reported numbers refer to the remaining 1180 benchmarks. Each run had a timeout of 120 seconds.

RQ1: Table 1 gives the experimental results for ItpTig+, SNA, and ItpTig with TPA as safety solver. ItpTig+ outperforms both SNA and ItpTig on terminating and on nonterminating instances, and solves 284 instances solved by neither SNA nor ItpTig; conversely, SNA and ItpTig each solve 7 instances not solved by ItpTig+. We attribute the improvement mainly to ItpTig+'s generation of focused sub-problems, which allows the analysis to concentrate on specific reachable states. that cover different regions of the reachable state space. In answer to **RQ1**, the combination of termination and nontermination reasoning in ItpTig+ solves substantially more instances than either SNA or ItpTig.

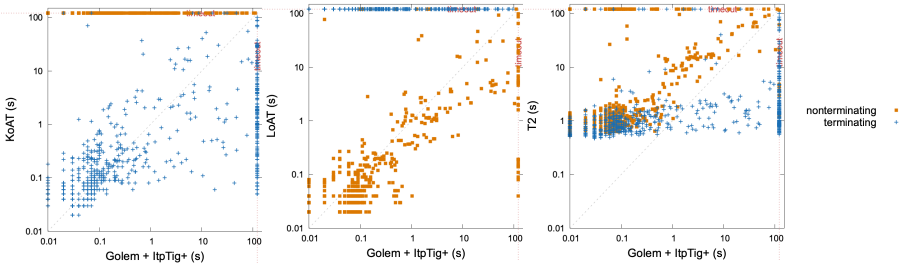


Fig. 5. Comparison of Golem + ItpTig+ with KoAT, LoAT and T2 (runtime in seconds).

Table 2. Comparison of Golem + ItpTig+ with state-of-the-art tools. *Unique*: instances solved by no other tool; for the two Golem configurations, only the non-Golem tools are taken into account.

Algorithm	Total	Terminating	Nonterminating	Unique
LoAT	525	0	525	24
KoAT	562	562	0	14
T2	1002	572	430	32
Golem	668	298	380	2
Golem + ItpTig+	906	405	501	10

The choice of the underlying safety verification engine matters: with TPA, ItpTig+ solves 805 instances (34 uniquely), and with PDKIND, 800 (29 uniquely).

RQ2: Termination tools typically combine several techniques in a portfolio, so we also evaluate ItpTig+ within Golem, whose previous portfolio combined lasso-shaped nontermination detection [4, 33] and a counter-based termination argument [19]. We added ItpTig+ with the TPA and PDKIND engines (Golem + ItpTig+). Figure 5 compares runtimes (points above the diagonal: Golem + ItpTig+ is faster), Table 2 gives the solved instances. Golem + ItpTig+ solves 906 benchmarks, 238 more than the previous Golem portfolio, against 1002 for T2, 562 for KoAT, and 525 for LoAT. On nontermination, it (501) is second only to LoAT (525); on termination (405), it is behind T2 (572) and KoAT (562). No unsound results were observed. Golem + ItpTig+ solves ten instances that no other compared tool solves. Two benchmarks, `juLinkedListCreateAddAll` and `juLinkedListCreateAddAllAt`, were not solved by any participant of the Termination Competition 2025, ItpTig+ proved their nontermination. In answer to **RQ2**, ItpTig+ substantially strengthens the Golem portfolio, making it second strongest on nontermination among the compared tools and complementary to them on termination.

6 Related Work

Nontermination. Nontermination analysis is typically centered on the construction of recurrent sets — sets of states from which execution cannot escape [8, 9, 11, 15, 16, 25]. These techniques range from template-based synthesis to safety-check-based recurrent set construction, the latter of which our approach directly extends [11]. An alternative approach detects nontermination by identifying lasso-shaped executions — finite paths leading to a cycle — via bounded model checking [4]. Leike and Heizmann [27] use geometric nontermination witnesses: finite representations of infinite executions via sums of geometric series. Unlike the approaches above, which treat nontermination in isolation, our technique is integrated with termination analysis within a unified framework, so that intermediate results guide one another.

Termination. The most widely studied approach to termination analysis is the synthesis of ranking functions [2, 15, 17, 24, 29, 31, 32, 37]. State-of-the-art techniques focus on the construction of multiphase or lexicographic ranking functions [2, 15], which significantly broaden the class of programs for which termination can be proved. However, ranking function synthesis is not always feasible. With fixpoint computation, [36] reduces termination and nontermination to the validity of a first-order fixpoint formula. Disjunctively well-founded transition invariants [12,23,30,34] enable over-approximating the behavior of the transition system. The central challenge there is the generation of the transition invariant itself: existing techniques rely either on template-based generation [34] or on the direct construction of a ranking relation from a particular trace [12]. In contrast, our approach uses Craig interpolation to guide transition invariant synthesis with respect to the actual termination conditions, producing property-directed invariants.

7 Conclusion

We have presented an interpolation-based approach to transition invariant generation for proving and disproving termination of infinite-state systems. The approach extends safety-based nontermination analysis with a termination proving capability, unifying the two within a single framework. Our key insight is to exploit terminating traces produced by the SNA as a source of structural information, using Craig interpolation to construct well-founded transition invariants guided by the termination conditions. The interaction between termination and nontermination reasoning is central to our technique: transition invariants focus the nontermination analysis on not covered states, while SNA provides terminating traces that drive invariant generation. We proved the soundness of the algorithm and conducted an extensive experimental evaluation. The results demonstrate the benefit of combining termination and nontermination analysis, and show that our approach achieves competitive performance against state-of-the-art tools, including the unique solution of instances never solved by any tool in the Termination Competition.

References

1. Ben-Amram, A.M., Doménech, J.J., Genaim, S.: Multiphase-linear ranking functions and their relation to recurrent sets. In: Chang, B.E. (ed.) Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings. pp. 459–480. Lecture Notes in Computer Science, Springer (2019). https://doi.org/10.1007/978-3-030-32304-2_22, https://doi.org/10.1007/978-3-030-32304-2_22
2. Ben-Amram, A.M., Genaim, S.: Ranking functions for linear-constraint loops. J. ACM **61**(4), 26:1–26:55 (2014). https://doi.org/10.1145/2629488, https://doi.org/10.1145/2629488
3. Beyer, D., Jankola, M., Lingsch-Rosenfeld, M.: Transition invariants revisited: Termination witnesses and their validation. In: Proc. CAV. Springer (2026), https://www.sosy-lab.org/research/pub/2026-CAV.Transition_Invariants_Revisited_Termination_Witnesses_and_Their_Validation.pdf
4. Biere, A., Artho, C., Schuppan, V.: Liveness checking as safety checking. In: Cleaveland, R., Garavel, H. (eds.) 7th International ERCIM Workshop in Formal Methods for Industrial Critical Systems, FMICS 2002, ICALP 2002 Satellite Workshop, Málaga, Spain, July 12-13, 2002. pp. 160–177. No. 2 in Electronic Notes in Theoretical Computer Science, Elsevier (2002). https://doi.org/10.1016/S1571-0661(04)80410-9, https://doi.org/10.1016/S1571-0661(04)80410-9
5. Blicha, M., Britikov, K., Sharygina, N.: The golem horn solver. In: Enea, C., Lal, A. (eds.) Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13965, pp. 209–223. Springer (2023), https://doi.org/10.1007/978-3-031-37703-7_10
6. Blicha, M., Fedyukovich, G., Hyvärinen, A.E.J., Sharygina, N.: Transition power abstractions for deep counterexample detection. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022. Lecture Notes in Computer Science, vol. 13243, pp. 524–542. Springer (2022), https://doi.org/10.1007/978-3-030-99524-9_29
7. Bradley, A.R., Manna, Z.: Property-directed incremental invariant generation. Formal Aspects Comput. **20**(4-5), 379–405 (2008). https://doi.org/10.1007/S00165-008-0080-9, https://doi.org/10.1007/s00165-008-0080-9
8. Brockschmidt, M., Cook, B., Ishtiaq, S., Khlaaf, H., Piterman, N.: T2: temporal property verification. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 387–393. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_22, https://doi.org/10.1007/978-3-662-49674-9_22
9. Brockschmidt, M., Ströder, T., Otto, C., Giesl, J.: Automated detection of non-termination and nullpointerexceptions for java bytecode. In: Beckert, B., Damiani, F., Gurov, D. (eds.) Formal Verification of Object-Oriented Software - International Conference, FoVeOOS 2011, Turin, Italy, October 5-7, 2011, Revised Selected Papers. pp. 123–141. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-31762-0_9, https://doi.org/10.1007/978-3-642-31762-0_9

10. Bruttomesso, R., Pek, E., Sharygina, N., Tsitovich, A.: The opensmt solver. In: Esparza, J., Majumdar, R. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 16th International Conference, TACAS 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, March 20-28, 2010. Proceedings. pp. 150–153. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-12002-2_12, https://doi.org/10.1007/978-3-642-12002-2_12
11. Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O'Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11
12. Cook, B., Podelski, A., Rybalchenko, A.: Abstraction refinement for termination. In: Hankin, C., Siveroni, I. (eds.) Static Analysis, 12th International Symposium, SAS 2005, London, UK, September 7-9, 2005, Proceedings. pp. 87–101. Lecture Notes in Computer Science, Springer (2005). https://doi.org/10.1007/11547662_8, https://doi.org/10.1007/11547662_8
13. Cook, B., Podelski, A., Rybalchenko, A.: Termination proofs for systems code. In: Schwartzbach, M.I., Ball, T. (eds.) Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation, Ottawa, Ontario, Canada, June 11-14, 2006. pp. 415–426. ACM (2006). https://doi.org/10.1145/1133981.1134029, https://doi.org/10.1145/1133981.1134029
14. Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. The Journal of Symbolic Logic **22**(3), 269–285 (1957)
15. Fedyukovich, G., Zhang, Y., Gupta, A.: Syntax-guided termination analysis. In: Chockler, H., Weissenbacher, G. (eds.) Computer Aided Verification - 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings, Part I. pp. 124–143. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-96145-3_7, https://doi.org/10.1007/978-3-319-96145-3_7
16. Frohn, F., Giesl, J.: Proving non-termination and lower runtime bounds with loat (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) Automated Reasoning - 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8-10, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13385, pp. 712–722. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_41, https://doi.org/10.1007/978-3-031-10769-6_41
17. Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, C., Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., Thiemann, R.: Analyzing program termination and complexity automatically with aprove. J. Autom. Reason. **58**(1), 3–31 (2017). https://doi.org/10.1007/S10817-016-9388-Y, https://doi.org/10.1007/s10817-016-9388-y
18. Giesl, J., Mesnard, F., Rubio, A., Thiemann, R., Waldmann, J.: Termination competition (termcomp 2015). In: Felty, A.P., Middeldorp, A. (eds.) Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings. pp. 105–108. Lecture Notes

in Computer Science, Springer (2015). https://doi.org/10.1007/978-3-319-21401-6_6, https://doi.org/10.1007/978-3-319-21401-6_6

19. Gulwani, S., Mehra, K.K., Chilimbi, T.M.: SPEED: precise and efficient static estimation of program computational complexity. In: Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009. pp. 127–139. ACM (2009). https://doi.org/10.1145/1480881.1480898
20. Heizmann, M., Hoenicke, J., Podelski, A.: Termination analysis by learning terminating programs. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 797–813. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_53, https://doi.org/10.1007/978-3-319-08867-9_53
21. Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: Opensmt2: An SMT solver for multi-core and cloud computing. In: Creignou, N., Berre, D.L. (eds.) Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9710, pp. 547–553. Springer (2016). https://doi.org/10.1007/978-3-319-40970-2_35, https://doi.org/10.1007/978-3-319-40970-2_35
22. Jovanovic, D., Dutertre, B.: Property-directed k-induction. In: Piskac, R., Talupur, M. (eds.) 2016 Formal Methods in Computer-Aided Design, FMCAD 2016, Mountain View, CA, USA, October 3-6, 2016. pp. 85–92. IEEE (2016). https://doi.org/10.1109/FMCAD.2016.7886665, https://doi.org/10.1109/FMCAD.2016.7886665
23. Kroening, D., Sharygina, N., Tsitovich, A., Wintersteiger, C.M.: Termination analysis with compositional transition invariants. In: Touili, T., Cook, B., Jackson, P.B. (eds.) Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. pp. 89–103. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_9, https://doi.org/10.1007/978-3-642-14295-6_9
24. Kura, S., Unno, H., Hasuo, I.: Decision tree learning in cegis-based termination analysis. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II. pp. 75–98. Lecture Notes in Computer Science, Springer (2021). https://doi.org/10.1007/978-3-030-81688-9_4, https://doi.org/10.1007/978-3-030-81688-9_4
25. Larraz, D., Nimkar, K., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Proving non-termination using max-smt. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 779–796. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_52, https://doi.org/10.1007/978-3-319-08867-9_52
26. Le, T.C., Qin, S., Chin, W.: Termination and non-termination specification inference. In: Grove, D., Blackburn, S.M. (eds.) Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015. pp. 489–498. ACM (2015). https://doi.org/10.1145/2737924.2737993, https://doi.org/10.1145/2737924.2737993

27. Leike, J., Heizmann, M.: Geometric nontermination arguments. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part II. pp. 266–283. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-89963-3_16, https://doi.org/10.1007/978-3-319-89963-3_16
28. Lommen, N., Giesl, J.: Aprove (koat + loat). In: Gurfinkel, A., Heule, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 205–211. Springer Nature Switzerland, Cham (2025)
29. Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings. pp. 239–251. Lecture Notes in Computer Science, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20
30. Podelski, A., Rybalchenko, A.: Transition invariants. In: 19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings. pp. 32–41. IEEE Computer Society (2004). https://doi.org/10.1109/LICS.2004.1319598, https://doi.org/10.1109/LICS.2004.1319598
31. Riley, D., Fedyukovich, G.: Exact loop bound analysis. Proc. ACM Program. Lang. **9**(PLDI), 1814–1837 (2025). https://doi.org/10.1145/3729323, https://doi.org/10.1145/3729323
32. Sarita, Y., Singh, A., Gomber, S., Singh, G., Vishwanathan, M.: Efficient ranking function-based termination analysis via bidirectional decompositional search. In: Krebbers, R. (ed.) Programming Languages and Systems - 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11-16, 2026, Proceedings, Part II. pp. 181–209. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-22723-2_7, https://doi.org/10.1007/978-3-032-22723-2_7
33. Schuppan, V., Biere, A.: Liveness checking as safety checking for infinite state spaces. In: Srba, J., Smolka, S.A. (eds.) Proceedings of the 7th International Workshop on Verification of Infinite-State Systems, INFINITY 2005, San Francisco, CA, USA, August 27, 2005. Electronic Notes in Theoretical Computer Science, vol. 149, pp. 79–96. Elsevier (2005). https://doi.org/10.1016/J.ENTCS.2005.11.018, https://doi.org/10.1016/j.entcs.2005.11.018
34. Tsitovich, A., Sharygina, N., Wintersteiger, C.M., Kroening, D.: Loop summarization and termination analysis. In: Abdulla, P.A., Leino, K.R.M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011. Proceedings. pp. 81–95. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_9, https://doi.org/10.1007/978-3-642-19835-9_9
35. Turing, A.M.: On computable numbers, with an application to the entscheidungsproblem. Proc. London Math. Soc. **s2-42**(1), 230–265 (1937). https://doi.org/10.1112/PLMS/S2-42.1.230, https://doi.org/10.1112/plms/s2-42.1.230

36. Unno, H., Terauchi, T., Gu, Y., Koskinen, E.: Modular primal-dual fixpoint logic solving for temporal verification. Proc. ACM Program. Lang. **7**(POPL), 2111–2140 (2023). https://doi.org/10.1145/3571265, `https://doi.org/10.1145/3571265`
37. Urban, C., Gurfinkel, A., Kahsai, T.: Synthesizing ranking functions from bits and pieces. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 54–70. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_4, `https://doi.org/10.1007/978-3-662-49674-9\_4`

A Proof of Lemma 3

Lemma 9. *Let $TS = \langle Init, Tr, X\rangle$ be a transition system and let $C(X)$ be a formula such that $\neg C(X)$ is unreachable in TS. If $TrInv(X, X')$ is a finite disjunction of formulas encoding well-founded relations and satisfies $\forall X, X', X''$:*

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$
$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

then TS is terminating.

Proof. We show that $TrInv$ is a disjunctively well-founded transition invariant over the reachable states of TS, from which termination follows by [30].

Disjunctive well-foundedness. By construction, for each disjunct d_i of $TrInv = \bigvee_i d_i$ there exists a ranking function. Therefore, every disjunct is a logical encoding of well-founded relation [29]. Hence $TrInv$ is disjunctively well-founded.

Transition invariant. We show that $\forall X, X'.\ R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $R(X)$ denotes the reachable states of TS, by induction on the number of steps n in $Tr^+(X, X')$.

Base case ($n = 1$): It is needed to prove that:

$$R(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

Since $\neg C(X)$ is unreachable, $R(X) \rightarrow C(X)$, and therefore $R(X) \wedge Tr(X, X') \rightarrow C(X) \wedge Tr(X, X')$.

The following assumption holds by the lemma statement:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X').$$

Therefore the base case holds.

Inductive step: Assume $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$ for some $n \geq 1$. Let $Tr^{n+1}(X, X'')$ hold, so there $\exists X, X', \ldots, X''.R(X) \wedge Tr^n(X, \ldots, X') \wedge Tr(X', X'')$. Since $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$, then trivially $R(X) \wedge Tr^n(X, \ldots, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X') \wedge Tr(X', X'')$. Now, due to the fact

that $R(X) \to C(X)$, the following holds: $R(X) \wedge \mathit{TrInv}(X, X') \wedge \mathit{Tr}(X', X'') \to C(X) \wedge \mathit{TrInv}(X, X') \wedge \mathit{Tr}(X', X'')$. By the lemma hypothesis:

$$C(X) \wedge \mathit{TrInv}(X, X') \wedge \mathit{Tr}(X', X'') \to \mathit{TrInv}(X, X'').$$

Therefore, through the chain of implications, we have:

$$R(X) \wedge \mathit{Tr}^{n+1}(X, X') \to \mathit{TrInv}(X, X').$$

Thus *TrInv* is a disjunctively well-founded transition invariant over all reachable states, and TS is terminating. □

B Correctness proofs

Proof of Lemma 4.

Proof. TERM is returned only at line 8. It means that there exists a transition invariant *TrInv* which is valid over states $C(X)$, and $\neg C(X)$ is unreachable from the initial states. The safety problem is updated iteratively, blocking the states that deterministically lead to termination (Lemma 2), so in the end the $P = \langle \mathit{Init}', \mathit{Tr}', \mathit{Sink}, X \rangle$, where $\mathit{Init}'(X) = \mathit{Init}(X) \wedge \bigwedge_{i=0}^{m} \neg S_i(X)$ and $\mathit{Tr}' = \mathit{Tr}(X, X') \wedge \bigwedge_{j=0}^{l} \neg S_j(X')$. According to Lemma 3, transition system $\langle \mathit{Init}', \mathit{Tr}', X \rangle$ is terminating, since *TrInv* is a disjunctively well-founded transition invariant over all reachable states.

Since the states satisfying $\bigvee_{i=0}^{m} S_i(X)$ and $\bigvee_{j=0}^{l} S_j(X')$ deterministically lead to the termination, every trace of $\langle \mathit{Init}, \mathit{Tr}, X \rangle$ either follows Tr' or passes through a blocked state, which leads to termination. Hence $\langle \mathit{Init}, \mathit{Tr}, X \rangle$ is terminating. □

Proof of Lemma 5.

Proof. The proof of this lemma follows directly from Lemma 1, as the nontermination analysis in Algorithm 4 follows exactly the same process as Algorithm 1. Added transition invariant synthesis flow does not influence safety checks and nontermination analysis. □

C Proof of Algorithm 5 invariant

Proof of Lemma 6.

Proof. Initially $B = P_0$. Since problems are pushed on top of B, B is popped only when it is the only element of *Prs*. The algorithm then either returns, or pushes as the new bottom (a) BlockTerm(B, ϕ) (line 10), which keeps (1) because Sink_B is unchanged and Tr_B only shrinks, and keeps (2) by Lemma 2; or (b) $\langle \mathit{Init}_B \wedge \neg \mathit{Sink}', \mathit{Tr}_B, \mathit{Sink}', X \rangle$ (line 23), which satisfies (1) by the definition of Sink' and keeps (2) because the removed initial states have no successors. □

D Proof that BlockTerm preserves the invariant

Proof of Lemma 2.

Proof. Let i be the index at which BlockTerm stops ($i = -1$ if it reaches line 7). For all $j > i$ the query at line 4 was unsatisfiable, so every Tr-successor of an S_j-state is in S_{j+1}, and $S_n \subseteq Sink$. Hence every Tr-path from a state in S_{i+1} reaches a state without successors within n steps. An infinite run therefore never visits S_{i+1}, so it uses no transition removed at line 5 and does not start in an initial state removed at line 7. □